Theme Switch (Light and Dark): Machine Coding Walkthrough

Problem: Build a theme switch for a web app: Light, Dark and System (follow the OS), remembered across visits, with no flash of the wrong theme on load.
Tech stack: React (Vite, JavaScript), plain CSS with custom properties (default, not given in the task)
Time limit: 60 minutes (default, not given in the task)

PART A: Thought process

Steps that work for any machine coding task:

1. **Clarify** the requirements. Write down what is in and out of scope before touching code.
2. **List the interactions.** Every user action and the expected result. This becomes your checklist.
3. **Sketch the architecture.** Components and who owns which state.
4. **Design the state.** Store the minimum, derive the rest.
5. **Name the hard parts** before you start, so you budget time for them.
6. **Build in layers.** A working ugly version first, then one working feature at a time.
7. **Handle states and edge cases.** Empty, error, blocked APIs, odd inputs.
8. **Wrap up.** Walk the interviewer through trade-offs and what you would add next.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	8 min
Basic toggle with CSS variables + persistence	12 min
System mode + live OS sync	8 min
No-flash inline script	7 min
Accessible switch UI + cross-tab sync	10 min
Edge cases, polish, testing by hand	6 min
Wrap-up and discussion	4 min

Think out loud. Say every decision and its reason as you make it ("I store the preference, not the resolved theme, because System has to keep following the OS"). The interviewer grades how you reason as much as the final code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- The user can choose **Light**, **Dark** or **System**.
- **System** follows the OS setting (`prefers-color-scheme`), and updates live if the OS changes while the page is open.
- A quick one-click **toggle** button flips between light and dark.
- The choice is **remembered** across reloads and visits.
- The correct theme shows **on the very first paint**: no white flash for dark users.
- Every part of the UI (backgrounds, text, borders, inputs, scrollbars) follows the theme.

Non-functional requirements

Area	What it means here
Performance	Switching the theme must not re-render the whole tree. Only swap CSS variable values on `<html>` ; the browser restyles, React does nothing.
Scalability	Adding a third theme (high contrast, sepia) should only mean a new CSS block and a new option.
Error handling	`localStorage` can throw (Safari private mode, blocked cookies, sandboxed iframes). The app must still work, just without memory. Bad stored values must not break it.
Accessibility	Keyboard usable, visible focus, screen reader labels, enough contrast in both themes, respect `prefers-reduced-motion` .
Reusability	A `ThemeProvider` + `useTheme()` hook that any component can use.
Extensibility	Easy to sync with a server profile later, or add more themes.

Clarifying questions to ask the interviewer

1. Do we need a **System** option, or only light/dark?
2. Should the choice persist? Only on this device (`localStorage`) or across devices (user profile on the server)?
3. Is this **client-rendered** (Vite SPA) or **server-rendered** (Next.js)? It changes how we avoid the flash.
4. Should other open tabs update when the theme changes in one tab?
5. Any design tokens given, or can I define my own color palette?
6. Should the switch animate (fade between themes)?

Assumptions we build with

- Three options: Light, Dark, System. **Default is System** for a first-time visitor.
- Persist on this device with `localStorage` under the key `theme-preference` .
- Client-rendered Vite app, so an **inline script in** `index.html` prevents the flash.
- Other tabs stay in sync.

- Colors are our own CSS custom properties; a short background/text fade that turns off with reduced motion.
- No UI library and no icon library (emoji icons, `aria-hidden`).

2. Identify interactions / use cases

Action / input	Expected behavior / output
First visit, OS is dark	Page paints dark immediately. Switch shows **System** selected.
First visit, OS is light	Page paints light. Switch shows **System**.
Click **Dark**	Theme becomes dark at once; `theme-preference = dark` saved.
Click **Light**	Theme becomes light; saved.
Click **System**	Theme follows the OS; `system` saved.
Click the toggle button	Flips the visible theme (light ↔ dark) and saves that explicit choice.
Reload after choosing Dark	Page paints dark on the first frame. No flash.
OS switches to dark while preference is System	Page turns dark live, no reload.
OS switches while preference is Light	Nothing changes (explicit choice wins).
Change theme in tab A	Tab B updates to the same preference.
Tab key / arrow keys on the switch	Focus ring visible; arrows move between Light / Dark / System.
Screen reader on the toggle	Reads "Switch to dark theme, button".
`localStorage` blocked	Theme still switches; just not remembered. No crash.
Stored value is garbage (`"blue"`)	Treated as System.
User prefers reduced motion	No fade animation.

3. Design the architecture

Module	Responsibility
`index.html` inline script	Sets `data-theme` on `<html>` before CSS and React load, so the first paint is correct.
`themeUtils.js`	Pure helpers: constants, `resolveTheme`, read/save preference safely, `applyTheme`.
`ThemeContext.js`	The React context object.
`ThemeProvider.jsx`	Owns the state, listens to the OS and other tabs, applies the theme to `<html>`.
`useTheme.js`	Hook to read the context, throws if used outside the provider.
`ThemeSwitch.jsx`	Three-way Light / Dark / System segmented control.
`ThemeToggle.jsx`	One-click light ↔ dark button.
`index.css`	Design tokens as CSS variables for each theme; components use only tokens.

Module	Responsibility
`App.jsx`	Demo page with cards and a form.

```
index.html (inline script sets data-theme before paint)
└── main.jsx
    └── ThemeProvider           (preference, systemTheme → theme)
        └── App                 (shows preference + resolved theme)
            ├── ThemeSwitch     (Light | Dark | System)  ── useTheme()
            ├── ThemeToggle     (🌞 / 🌙)                ── useTheme()
            └── Cards, Form     (use CSS variables only, no theme logic)
```

Start simple: the first version is one `useState` and a class on `<html>`. Only split out `themeUtils.js` when the inline script, the provider and the tests all need the same rules.

4. Design the state / data model

State or field	Purpose
`preference` (`'light' | 'dark' | 'system'`)	What the **user chose**. This is what we store.
`systemTheme` (`'light' | 'dark'`)	What the **OS currently says**. Updated by a `matchMedia` listener.
`theme` (derived)	What is **on screen**: `preference === 'system' ? systemTheme : preference`.
`localStorage['theme-preference']`	Persists `preference` across visits.
`<html data-theme>`	The single DOM switch the CSS reads.

```
const [preference, setPreferenceState] = useState(readPreference); // lazy init: read storage once
const [systemTheme, setSystemTheme] = useState(getSystemTheme);
const theme = resolveTheme(preference, systemTheme); // derived, never stored
```

Senior note

- **Store the preference, derive the theme.** If you store only `'dark'`, you can't tell "user chose dark" from "OS is dark", and System mode breaks the moment the OS changes.
- `theme` is derived on every render: no `useEffect` to keep two states in sync, so they can never disagree.
- **Ownership:** `ThemeProvider` owns both states. Components only read `useTheme()` and call `setPreference` / `toggleTheme`. The DOM attribute is a side effect owned by one effect in the provider.
- Components don't need `theme` to style themselves: CSS variables do that. Only UI that *shows* the theme (the switch, an icon, a chart library) reads it.

5. Design the data structures and contracts

Example data

```
export const STORAGE_KEY = 'theme-preference';
export const PREFERENCES = ['light', 'dark', 'system'];

const OPTIONS = [
  { value: 'light', label: 'Light', icon: '🌞' },
  { value: 'dark', label: 'Dark', icon: '🌙' },
  { value: 'system', label: 'System', icon: '💻' },
];
```

The `value` is a stable id: it is the storage value, the radio value and the React `key`. Labels can be translated later without touching stored data.

Contracts

`useTheme()` returns

```
{
  preference: 'light' | 'dark' | 'system', // the user's choice
  theme: 'light' | 'dark',                 // what is on screen
  setPreference: (next) => void,           // ignores invalid values
  toggleTheme: () => void,                 // flips the visible theme, saves it
}
```

Storage: `localStorage.setItem('theme-preference', 'dark')`. Anything else stored is treated as `'system'`.

DOM contract with CSS: `<html data-theme="dark" style="color-scheme: dark">`. CSS only reads `[data-theme='dark']`.

CSS tokens: `--bg`, `--surface`, `--text`, `--muted`, `--border`, `--accent`, `--accent-contrast`, `--focus`. Components never use raw colors.

There is no backend API. If one is added later, the contract would be `PATCH /me/preferences { theme: 'dark' }`, with `localStorage` as the fast local cache.

6. Trace the data flow

User clicks "Dark" in tab A while the OS is light

```
Click "Dark" radio
  │
  ▼
onChange → setPreference('dark')
  │   ├── isValidPreference('dark') ✓
  │   ├── setPreferenceState('dark')            (React state)
  │   └── localStorage.setItem('theme-preference','dark')   (persist)
  ▼
ThemeProvider re-renders
```

```
  |   theme = resolveTheme('dark', 'light') → 'dark'   (derived)
  ▼
useEffect([theme]) → applyTheme('dark')
  |   <html data-theme="dark" style="color-scheme: dark">
  ▼
CSS: :root[data-theme='dark'] swaps the variable values
  |   browser restyles every element; no component re-renders for color
  ▼
Output: page is dark, "Dark" radio checked, status "Preference: dark · Showing: dark"
  |
  ▼  (side effect in other tabs)
Tab B gets a 'storage' event { key: 'theme-preference', newValue: 'dark' }
  → setPreferenceState('dark') → same flow, without saving again
```

7. Important technical challenges

Challenge 1: Flash of the wrong theme on load (FOUC)

Problem. A dark-mode user reloads. The HTML and CSS paint first with the default light theme. React loads 200 ms later, reads `localStorage`, and switches to dark. The user sees a white flash on every visit.

Solution. Decide the theme **before the first paint**, in a tiny blocking inline script in `<head>`. It runs before the CSS bundle and React. It must repeat the same rules as `readPreference` + `resolveTheme`, and be wrapped in `try/catch` because storage can throw.

```
<meta name="color-scheme" content="light dark" />
<script>
  (function () {
    try {
      var pref = localStorage.getItem('theme-preference');
      var dark = pref === 'dark' ||
        (pref !== 'light' && window.matchMedia('(prefers-color-scheme: dark)').matches);
      var theme = dark ? 'dark' : 'light';
      document.documentElement.dataset.theme = theme;
      document.documentElement.style.colorScheme = theme;
    } catch (e) { /* storage blocked: CSS default (light) applies */ }
  })();
</script>
```

React then starts with the same value from `readPreference`, so its first `applyTheme` writes what is already there: nothing changes on screen.

Challenge 2: System mode that stays live

Problem. The user picks System at 6 pm, the OS switches to dark at sunset. If we only read `prefers-color-scheme` once, the page stays light. And if we store the resolved `'dark'`, we lose the fact that the user wanted System.

Solution. Keep `preference` and `systemTheme` as two separate states, listen to the media query, and derive the visible theme.

```
useEffect(() => {
  if (!window.matchMedia) return;
  const mql = window.matchMedia(SYSTEM_DARK_QUERY);
  const onChange = (e) => setSystemTheme(e.matches ? 'dark' : 'light');
  mql.addEventListener('change', onChange);
  return () => mql.removeEventListener('change', onChange); // no leak on unmount
}, []);

const theme = resolveTheme(preference, systemTheme);
```

An explicit Light or Dark choice ignores `systemTheme` automatically, because `resolveTheme` only uses it for `'system'`.

Challenge 3: Theming the whole UI without re-rendering it

Problem. Passing `theme` into every component (`className={theme === 'dark' ? ...}`) means every component re-renders on switch, and every new component must remember to handle it. Native parts (scrollbars, date pickers, autofill) still stay light.

Solution. One attribute on `<html>`, **CSS custom properties** for every color, and `color-scheme` for native controls.

```
:root { --bg: #f6f7fb; --text: #1c1d26; --accent: #4f46e5; }
:root[data-theme='dark'] { --bg: #0f1117; --text: #e7e8ef; --accent: #8b93ff; }

body { background: var(--bg); color: var(--text); }
```

```
export function applyTheme(theme) {
  const root = document.documentElement;
  root.dataset.theme = theme;
  root.style.colorScheme = theme; // scrollbars + form controls follow
}
```

Challenge 4: Robust persistence and cross-tab sync

Problem. Safari private mode throws on `localStorage.setItem`; an old version of the app stored `"blue"`; the user has two tabs open and changes the theme in one.

Solution. Wrap storage in `try/catch`, validate everything read, and listen to the `storage` event (fires only in the *other* tabs). Save inside the setter, not in an effect, so a tab that receives a sync does not write back.

```
export function readPreference() {
  try {
    const value = localStorage.getItem(STORAGE_KEY);
    return isValidPreference(value) ? value : 'system';
  } catch {
    return 'system';
  }
}

useEffect(() => {
  const onStorage = (e) => {
    if (e.key !== STORAGE_KEY) return;
```

```
    setPreferenceState(isValidPreference(e.newValue) ? e.newValue : 'system');
  };
  window.addEventListener('storage', onStorage);
  return () => window.removeEventListener('storage', onStorage);
}, []);
```

8. Implement in the correct order

#	Step	Working result after the step
1	CSS tokens for light, all styles use `var(--…)`	Page styled with tokens
2	Dark token block on `:root[data-theme='dark']`	Setting the attribute by hand in DevTools flips the theme
3	`ThemeProvider` with `useState('light')` + effect writing `data-theme`, plus a toggle button	Working light/dark toggle
4	`useTheme()` hook with the outside-provider guard	Any component can read the theme
5	Persist to `localStorage` with lazy init	Choice survives reload (with a flash)
6	Add `'system'` preference, `systemTheme` state, derived `theme`	Three modes work
7	`matchMedia` change listener	System mode follows the OS live
8	Inline script in `index.html` + `color-scheme`	No flash on reload
9	`ThemeSwitch` with native radios, `sr-only` legend, focus styles	Accessible three-way switch
10	`try/catch` + validation in storage helpers	Survives private mode and bad data
11	`storage` event listener	Tabs stay in sync
12	Fade transition + `prefers-reduced-motion`, final manual test	Polished

9. Cover all states and edge cases

State / edge case	Handling
Initial, no stored value	Preference `system`, theme from the OS.
Initial, stored `dark`	Inline script paints dark; React agrees on first render.
Loading (before JS)	Inline script already set the theme; CSS default is light if JS is disabled.
Success (switched)	Attribute updated, value saved, radio checked, status text updated.
Empty / invalid storage value	Treated as `system`.
Error: `localStorage` throws on read	`readPreference` returns `system`.
Error: `localStorage` throws on write	Choice works for this session only.

State / edge case	Handling
`matchMedia` not supported	`getSystemTheme` returns `light`; listener is skipped.
OS changes, preference System	Theme follows live.
OS changes, preference Light/Dark	Ignored.
Change in another tab	`storage` event updates this tab.
Storage cleared in another tab (`newValue = null`)	Falls back to `system`.
`useTheme` used outside provider	Throws a clear error during development.
Reduced motion	Transition disabled.
Rapid clicks on the toggle	Each click is a plain state update; no async race.
Native inputs and scrollbars	Follow via `color-scheme`.

10. Final structure

```
_43_theme_switch_light_and_dark/
├── index.html                # inline no-flash script + color-scheme meta
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx              # wraps App in ThemeProvider
    ├── App.jsx               # demo page
    ├── index.css             # tokens per theme + component styles
    ├── theme/
    │   ├── themeUtils.js     # pure helpers + constants
    │   ├── ThemeContext.js   # createContext
    │   ├── ThemeProvider.jsx # state, listeners, applyTheme
    │   └── useTheme.js       # context hook with guard
    └── components/
        ├── ThemeSwitch.jsx   # Light | Dark | System
        └── ThemeToggle.jsx   # one-click light ↔ dark
```

`ThemeContext.js` is its own file so `ThemeProvider.jsx` exports only a component (keeps React Fast Refresh happy).

11. Full working code

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="color-scheme" content="light dark" />
    <title>Theme Switch</title>
```

```
    <!-- Runs before CSS and React: sets the theme before first paint, so no flash of the wrong theme
-->
    <script>
      (function () {
        try {
          var pref = localStorage.getItem('theme-preference');
          var dark = pref === 'dark' ||
            (pref !== 'light' && window.matchMedia('(prefers-color-scheme: dark)').matches);
          var theme = dark ? 'dark' : 'light';
          document.documentElement.dataset.theme = theme;
          document.documentElement.style.colorScheme = theme;
        } catch (e) { /* storage blocked: CSS default (light) applies */ }
      })();
    </script>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

src/theme/themeUtils.js

```
export const STORAGE_KEY = 'theme-preference';
export const PREFERENCES = ['light', 'dark', 'system'];
export const SYSTEM_DARK_QUERY = '(prefers-color-scheme: dark)';

export const isValidPreference = (value) => PREFERENCES.includes(value);

// 'system' follows the OS; 'light' / 'dark' are explicit user choices
export const resolveTheme = (preference, systemTheme) =>
  preference === 'system' ? systemTheme : preference;

export function readPreference() {
  try {
    const value = localStorage.getItem(STORAGE_KEY);
    return isValidPreference(value) ? value : 'system';
  } catch {
    return 'system'; // storage blocked (private mode, sandboxed iframe)
  }
}

export function savePreference(preference) {
  try {
    localStorage.setItem(STORAGE_KEY, preference);
  } catch {
    // storage blocked: the choice still works for this session, it just is not remembered
  }
}

export const getSystemTheme = () =>
  window.matchMedia?.(SYSTEM_DARK_QUERY).matches ? 'dark' : 'light';

export function applyTheme(theme) {
  const root = document.documentElement;
  root.dataset.theme = theme;
```

```
  root.style.colorScheme = theme; // native scrollbars and form controls follow the theme
}
```

src/theme/ThemeContext.js

```
import { createContext } from 'react';

export const ThemeContext = createContext(null);
```

src/theme/ThemeProvider.jsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { ThemeContext } from './ThemeContext';
import {
  STORAGE_KEY,
  SYSTEM_DARK_QUERY,
  applyTheme,
  getSystemTheme,
  isValidPreference,
  readPreference,
  resolveTheme,
  savePreference,
} from './themeUtils';

export default function ThemeProvider({ children }) {
  const [preference, setPreferenceState] = useState(readPreference);
  const [systemTheme, setSystemTheme] = useState(getSystemTheme);

  // derived, never stored: the theme actually on screen
  const theme = resolveTheme(preference, systemTheme);

  // follow OS light/dark changes live (matters only while preference is 'system')
  useEffect(() => {
    if (!window.matchMedia) return;
    const mql = window.matchMedia(SYSTEM_DARK_QUERY);
    const onChange = (e) => setSystemTheme(e.matches ? 'dark' : 'light');
    mql.addEventListener('change', onChange);
    return () => mql.removeEventListener('change', onChange);
  }, []);

  // keep other open tabs in sync ('storage' fires only in the other tabs)
  useEffect(() => {
    const onStorage = (e) => {
      if (e.key !== STORAGE_KEY) return;
      setPreferenceState(isValidPreference(e.newValue) ? e.newValue : 'system');
    };
    window.addEventListener('storage', onStorage);
    return () => window.removeEventListener('storage', onStorage);
  }, []);

  useEffect(() => {
    applyTheme(theme);
  }, [theme]);

  const setPreference = useCallback((next) => {
    if (!isValidPreference(next)) return;
```

```
    setPreferenceState(next);
    savePreference(next); // saved here, not in an effect, so a synced tab does not write back
  }, []);

  const toggleTheme = useCallback(() => {
    setPreference(theme === 'dark' ? 'light' : 'dark');
  }, [theme, setPreference]);

  const value = useMemo(
    () => ({ preference, theme, setPreference, toggleTheme }),
    [preference, theme, setPreference, toggleTheme],
  );

  return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>;
}
```

src/theme/useTheme.js

```
import { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

export default function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme must be used inside <ThemeProvider>');
  return ctx;
}
```

src/components/ThemeSwitch.jsx

```
import useTheme from '../theme/useTheme';

const OPTIONS = [
  { value: 'light', label: 'Light', icon: '☀️' },
  { value: 'dark', label: 'Dark', icon: '🌙' },
  { value: 'system', label: 'System', icon: '💻' },
];

// native radios: arrow keys, focus and screen reader "1 of 3" come for free
export default function ThemeSwitch() {
  const { preference, setPreference } = useTheme();

  return (
    <fieldset className="theme-switch">
      <legend className="sr-only">Theme</legend>
      {OPTIONS.map((opt) => (
        <label key={opt.value} className="theme-switch__option">
          <input
            type="radio"
            name="theme"
            value={opt.value}
            checked={preference === opt.value}
            onChange={() => setPreference(opt.value)}
          />
          <span aria-hidden="true">{opt.icon}</span>
          {opt.label}
        </label>
```

```
      ))}
    </fieldset>
  );
}
```

src/components/ThemeToggle.jsx

```
import useTheme from '../theme/useTheme';

export default function ThemeToggle() {
  const { theme, toggleTheme } = useTheme();
  const next = theme === 'dark' ? 'light' : 'dark';

  return (
    <button
      type="button"
      className="theme-toggle"
      onClick={toggleTheme}
      aria-label={`Switch to ${next} theme`}
      title={`Switch to ${next} theme`}
    >
      <span aria-hidden="true">{theme === 'dark' ? '🌞' : '🌙'}</span>
    </button>
  );
}
```

src/App.jsx

```
import ThemeSwitch from './components/ThemeSwitch';
import ThemeToggle from './components/ThemeToggle';
import useTheme from './theme/useTheme';

const CARDS = [
  { id: 'c1', title: 'CSS variables', body: 'Every color is a token; the theme only swaps token
values.' },
  { id: 'c2', title: 'No flash', body: 'An inline script sets the theme before the first paint.' },
  { id: 'c3', title: 'System aware', body: 'System mode follows the OS setting live, even while open.'
},
];

export default function App() {
  const { preference, theme } = useTheme();

  return (
    <div className="app">
      <header className="header">
        <h1>Theme Switch</h1>
        <div className="header__actions">
          <ThemeSwitch />
          <ThemeToggle />
        </div>
      </header>

      <main className="content">
        <p className="status" aria-live="polite">
          Preference: <strong>{preference}</strong> · Showing: <strong>{theme}</strong>
```

```
        </p>

        <div className="grid">
          {CARDS.map((card) => (
            <article key={card.id} className="card">
              <h2>{card.title}</h2>
              <p>{card.body}</p>
            </article>
          ))}
        </div>

        <form className="card form" onSubmit={(e) => e.preventDefault()}>
          <label htmlFor="email">Email</label>
          <input id="email" type="email" placeholder="you@example.com" />
          <button type="submit" className="btn">Subscribe</button>
        </form>
      </main>
    </div>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import ThemeProvider from './theme/ThemeProvider';
import App from './App';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </StrictMode>,
);
```

src/index.css

```
/* light is the default, so the page is readable even if JS never runs */
:root {
  --bg: #f6f7fb;
  --surface: #ffffff;
  --text: #1c1d26;
  --muted: #5c5f70;
  --border: #dfe1ea;
  --accent: #4f46e5;
  --accent-contrast: #ffffff;
  --focus: #2563eb;
}

:root[data-theme='dark'] {
  --bg: #0f1117;
  --surface: #1a1d27;
  --text: #e7e8ef;
  --muted: #a3a6b5;
```

```
  --border: #2d3142;
  --accent: #8b93ff;
  --accent-contrast: #0f1117;
  --focus: #93c5fd;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
  background: var(--bg);
  color: var(--text);
  transition: background-color 0.2s ease, color 0.2s ease;
}

@media (prefers-reduced-motion: reduce) {
  body { transition: none; }
}

.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  overflow: hidden;
  clip: rect(0 0 0 0);
  white-space: nowrap;
}

.app { max-width: 960px; margin: 0 auto; padding: 16px; }

.header {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  align-items: center;
  justify-content: space-between;
  padding-bottom: 16px;
  border-bottom: 1px solid var(--border);
}

.header h1 { margin: 0; font-size: 1.4rem; }
.header__actions { display: flex; gap: 8px; align-items: center; }

.theme-switch {
  display: inline-flex;
  margin: 0;
  padding: 3px;
  border: 1px solid var(--border);
  border-radius: 999px;
  background: var(--surface);
}

.theme-switch__option {
  display: inline-flex;
  gap: 6px;
  align-items: center;
  padding: 6px 12px;
  border-radius: 999px;
  color: var(--muted);
```

```
  cursor: pointer;
  font-size: 0.9rem;
}

/* keep the radio focusable but invisible; the label is the visual control */
.theme-switch__option input {
  position: absolute;
  opacity: 0;
  pointer-events: none;
}

.theme-switch__option:has(input:checked) {
  background: var(--accent);
  color: var(--accent-contrast);
}

.theme-switch__option:has(input:focus-visible),
.theme-toggle:focus-visible,
.btn:focus-visible,
.form input:focus-visible {
  outline: 2px solid var(--focus);
  outline-offset: 2px;
}

.theme-toggle {
  width: 40px;
  height: 40px;
  border: 1px solid var(--border);
  border-radius: 50%;
  background: var(--surface);
  font-size: 1.1rem;
  cursor: pointer;
}

.content { padding-top: 16px; }
.status { color: var(--muted); }

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
}

.card {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: var(--surface);
}

.card h2 { margin: 0 0 8px; font-size: 1.05rem; }
.card p { margin: 0; color: var(--muted); }

.form {
  display: flex;
  flex-direction: column;
  gap: 8px;
  max-width: 360px;
  margin-top: 16px;
}
```

```
.form input {
  padding: 8px 10px;
  border: 1px solid var(--border);
  border-radius: 8px;
  background: var(--bg);
  color: var(--text);
}

.btn {
  padding: 8px 14px;
  border: none;
  border-radius: 8px;
  background: var(--accent);
  color: var(--accent-contrast);
  cursor: pointer;
}
```

Quick logic check (node)

```
import assert from 'node:assert/strict';
import { isValidPreference, resolveTheme } from './src/theme/themeUtils.js';

assert.equal(resolveTheme('system', 'dark'), 'dark');
assert.equal(resolveTheme('system', 'light'), 'light');
assert.equal(resolveTheme('light', 'dark'), 'light');
assert.equal(resolveTheme('dark', 'light'), 'dark');
assert.ok(isValidPreference('system'));
assert.ok(!isValidPreference('blue'));
assert.ok(!isValidPreference(null));
console.log('selfcheck ok');
```

12. Wrap-up

Edge cases handled

- No flash on reload, for both stored choices and System.
- System mode follows OS changes live; explicit choices ignore the OS.
- Blocked or broken `localStorage`, invalid stored values, missing `matchMedia`.
- Cross-tab sync without write-back loops.
- Keyboard and screen reader support, visible focus, reduced motion, native controls themed via `color-scheme`.

Trade-offs to mention

- **Inline script duplicates the resolve rule.** It's 8 lines that must stay in sync with `themeUtils.js`. The alternative (a flash) is worse. A build step could inject it from one source.
- `localStorage` **is per device.** Cross-device sync needs a user profile on the server.
- `data-theme` **attribute vs a class.** Same thing; the attribute holds exactly one value, so no "both light and dark" bug.

- **Native radios vs custom `role="radiogroup"`.** Native gives keyboard and a11y for free; the cost is a little CSS to hide the input.
- **`:has()` for the checked style** is supported in all current browsers; for older ones, add a class on the label from React.
- **Context value is memoized**, but `theme` changes still re-render consumers. That's only the switch and toggle, because everything else uses CSS variables.

What to add with more time

- Unit tests for `resolveTheme` / storage helpers, and React Testing Library tests that mock `matchMedia` and dispatch `storage` events.
- A Playwright test that reloads with a stored dark value and checks the first screenshot is dark.
- More themes (high contrast) and contrast checks for every token pair.
- Server sync of the preference for signed-in users, and SSR support (cookie read on the server).
- Smooth theme transition with the View Transitions API.

13. Counter questions the interviewer may ask

Requirements

Q1. Why add a System option at all? The task said light and dark.

Many users set dark mode at the OS level and expect sites to follow it, including automatic sunset switching. Defaulting to System means a first-time visitor gets the right theme with no click. It costs one extra state and a media listener.

Q2. What should the default be for a first-time visitor?

System. It respects what the user already told their OS. Defaulting to light would flash-blind dark-mode users; defaulting to dark ignores light-mode users.

State design

Q3. Why store `preference` and not just the resolved `theme`?

If you store `'dark'`, you can't tell "user chose dark" from "OS was dark when they chose System". Then System mode can't follow future OS changes. Store the intent, derive what is shown.

Q4. Why not keep `theme` in its own `useState` and sync it with an effect?

Two states that must agree will disagree for one render (and you'd paint the wrong theme for a frame). A derived value computed during render can never be out of sync and needs no effect.

Q5. Why Context and not Redux / Zustand?

It's one small, rarely changing value. Context is built in and enough. If the app already uses a store, the theme can live there; the CSS-variable approach stays the same.

Q6. Won't Context re-render the whole app when the theme changes?

Only components that call `useTheme()` re-render: here, the switch, the toggle and the status line. Everything else is restyled by the browser through CSS variables, with zero React work. The context value is also memoized so unrelated provider renders don't cascade.

Challenge 1: no flash

Q7. Why not just use `useLayoutEffect` to apply the theme?

`useLayoutEffect` runs before paint of *React's* first render, but the browser has already painted the HTML and CSS before the JS bundle even downloads. Only a blocking inline script in `<head>` runs early enough.

Q8. How would you do this in Next.js / SSR?

Read the preference from a **cookie** on the server and render `<html data-theme="dark">` directly. For System (unknown on the server), still include the inline script, and add `suppressHydrationWarning` on `<html>` because the attribute may differ from the server HTML.

Q9. What if JavaScript is disabled?

The CSS default (light) applies. You can also add a `@media (prefers-color-scheme: dark)` block on `:root:not([data-theme])` so no-JS users still get their OS theme, at the cost of duplicating the dark tokens.

Challenge 2: System mode

Q10. Why `addEventListener('change')` and not `mql.addListener`?

`addListener` is deprecated. `addEventListener` works in all modern browsers. For Safari below 14 you would fall back to `addListener` / `removeListener`.

Q11. Do you need to clean up the listener?

Yes. Without the cleanup, unmounting the provider (or Strict Mode's double mount in development) leaves a listener that calls `setState` on a dead component and leaks memory.

Challenge 3: CSS variables

Q12. Why CSS variables and not two separate stylesheets or CSS-in-JS theme objects?

Variables switch instantly with one attribute change, no new network request, no re-render, and work inside any CSS. Swapping stylesheets can flash while loading; CSS-in-JS theme props force React re-renders.

Q13. What does `color-scheme` do?

It tells the browser which palette to use for things you don't style yourself: scrollbars, default form controls, autofill backgrounds, the canvas color. Without it, a dark page can still show a bright white scrollbar.

Q14. What about images and charts in dark mode?

Use `<picture><source media="(prefers-color-scheme: dark)">` only for System; for an explicit choice, pick the image source from `theme` in React. Chart libraries take colors as props, so read the CSS variables or pass `theme` to them.

Challenge 4: persistence and sync

Q15. Why wrap `localStorage` in try/catch?

It throws in some private modes, when storage is full, when cookies are blocked, and in sandboxed iframes. An uncaught error at startup would leave the user with a blank page, over a cosmetic feature.

Q16. Why does the `storage` event not cause an infinite loop between tabs?

The event fires only in *other* tabs, and the receiving tab updates state without calling `savePreference`. Saving happens in the user-facing setter, not in an effect on `preference`.

Q17. What if the user is logged in and expects the same theme on their phone?

Save the preference to the user profile (`PATCH /me/preferences`) and keep `localStorage` (or a cookie) as the fast local cache for the first paint. On login, the server value wins and updates the cache.

Implementation steps and accessibility

Q18. Why native radio inputs for the switch instead of buttons?

It's a "pick one of three" control, which is exactly a radio group. Native radios give arrow-key navigation, one tab stop, and "Light, radio button, 1 of 3" announcements for free.

Q19. How do you label the icon-only toggle?

`aria-label="Switch to dark theme"` describes the action; the emoji is `aria-hidden`. Alternatively use `aria-pressed` with a fixed label "Dark mode". Don't do both: the label would contradict the state.

Q20. How do you check accessibility of the colors?

Check every text/background token pair for WCAG AA contrast (4.5:1 for body text) in both themes, and make the focus color visible on both backgrounds. That's why `--focus` is a token too.

Performance and "what if" twists

Q21. What if the app has 10,000 DOM nodes? Is switching slow?

Changing one attribute makes the browser recalculate styles once, which is fast even for big pages. Avoid `transition: all` on every element: animate only `background-color` and `color` on a few containers, or briefly disable transitions during the switch.

Q22. What if the design team wants 5 themes?

Add one CSS block per theme (`[data-theme='sepia']`), extend `PREFERENCES` and `OPTIONS` , and make the toggle cycle or remove it. `resolveTheme` and the provider stay the same.

Q23. What if a part of the page must always be dark (e.g. a video player)?

Set `data-theme="dark"` on that container and scope the token block as `[data-theme='dark']` instead of `:root[data-theme='dark']` . Variables cascade, so the subtree uses the dark tokens.

Trade-offs

Q24. Isn't duplicating logic in the inline script a code smell?

Yes, a small one, and worth it. It's the only code that can run before paint. Keep it tiny, and either generate it from `themeUtils.js` at build time or cover both with the same test cases.

Q25. How would you test this?

Unit-test `resolveTheme` and storage helpers (done with `node:assert` here). With React Testing Library: mock `window.matchMedia` , fire its `change` listener, dispatch a `StorageEvent` , and assert `document.documentElement.dataset.theme` . With Playwright: set storage, reload, and check the first screenshot is dark.